



8. Memory Access and Performance Optimization

- **Key Rule:** Use `#pragma no_alias` when you want to improve the performance of memory accesses, especially for large data arrays or matrices.
- **Reason:** By ensuring no aliasing, the compiler can optimize memory access patterns and reduce cache contention, leading to better performance.
- **Example:**

c

Copy code

```
#pragma no_alias
for (int i = 0; i < N; i++) {
    A[i] = B[i] + C[i]; // No aliasing between arrays A, B, and C
}
```

Example of Correct Usage:

c

Copy code

```
#define N 1024

void optimized_update(int *restrict A, int *restrict B, int *restrict C) {
    #pragma no_alias
    for (int i = 0; i < N; i++) {
        A[i] = B[i] + C[i]; // No aliasing, arrays A, B, and C do not overlap
    }
}
```

- In this example:
 - **No Aliasing:** `A`, `B`, and `C` are distinct arrays, so `#pragma no_alias` is applied

Message ChatGPT



